# ProRisk Manager AI - Developer Blueprint

**Version:** 0.2 (2026-01-28)
**Date:** 2026-02-18

---

## 1. Project Overview

**ProRisk Manager AI** is a React-based web application designed for project risk management. It enables Project Managers (PMs) and teams to identify, assess, and mitigate risks across various industries (Power Plants, Petrochemical, etc.).

**Key Capabilities:**

- **Risk Register Management:** CRUD operations for risk items.
- **Quantitative Assessment:** Impact vs. Likelihood scoring (Initial vs. Residual Risk).
- **AI Integration:** Uses Groq (Llama-3) to suggest mitigation strategies based on risk descriptions.
- **Visual Dashboards:** Heatmaps (Risk Matrix), Bar charts, and Status tracking.
- **Baseline Comparison:** Compares project risks against industry baselines.
- **User Management:** Role-based access control (Admin vs. User) powered by Firebase.

---

## 2. Technology Stack

### Core Frontend

- **Framework:** React 19 (https://react.dev/)
- **Build Tool:** Vite (https://vitejs.dev/)
- **Language:** TypeScript (https://www.typescriptlang.org/)
- **Styling:** Tailwind CSS (https://tailwindcss.com/)
- **Icons:** Lucide React (https://lucide.dev/)

### Backend & Services

- **BaaS (Backend-as-a-Service):** Firebase (https://firebase.google.com/)
    - **Authentication:** Email/Password via Firebase Auth.
    - **Database:** Cloud Firestore (NoSQL).
    - **Hosting:** Firebase Hosting.
- **AI Service:** Groq API (https://groq.com/)
    - **Model:** `llama-3.3-70b-versatile`
    - **Function:** Generates risk mitigation strategies and action plans.

### Utilities

- **CSV Processing:** `papaparse` (Importing/Exporting risk data).
- **Charts:** Custom SVG/Div based implementations (found in `components/`).

---

## 3. Project Structure

The project currently uses a flat structure in the root for source files, which may be refactored into a `src/` directory in future iterations.

```
/
├── .env.local                  # Environment variables (API Keys)
├── .firebaserc & firebase.json # Firebase configuration
├── App.tsx                     # Main Application Logic (Routing, State, Dashboard)
├── index.html                  # Entry HTML
├── index.tsx                   # React Entry Point
├── types.ts                    # Global TypeScript Interfaces (Data Models)
├── vite.config.ts              # Vite Configuration
├── components/                 # UI Components
│   ├── AdminPanel.tsx          # User management interface
│   ├── ProjectForm.tsx         # Project creation/editing
│   ├── RiskForm.tsx            # Risk creation/editing (Modal)
│   ├── RiskMatrix.tsx          # 5x5 Heatmap Visualization
│   ├── ... (Charts, Modals, Auth Screens)
├── services/                   # Business Logic & API Wrappers
│   ├── firebaseService.ts      # Firestore & Auth methods
│   ├── groqService.ts          # AI API integration
│   ├── riskBaselineService.ts  # Industry baseline logic
│   └── adminService.ts         # User administration logic
└── templates/                  # Static templates (e.g., CSV Import Guide)
```

---

## 4. Key Modules & Features

### A. Authentication & User Profile (`App.tsx`, `firebaseService.ts`)

- **Login:** Controlled by `LoginPage.tsx`.
- **Profile:** Users have roles (`Admin`, `User`) stored in Firestore `users/{uid}`.
- **Password Change:** Enforced for first-time logins via `ChangePasswordScreen.tsx`.
- **Access Control:** Users can only see/edit projects assigned to them (unless Admin).

### B. Risk Management (`RiskForm.tsx`, `RiskMatrix.tsx`)

- **Data Model:** `RiskItem` (defined in `types.ts`).
- **Versioning:** Risks maintain a `history` array of `RiskSnapshot` to track changes over time.
- **Scoring:** 5x5 Matrix (Impact x Likelihood).
    - **Initial Risk:** Pre-mitigation score.
    - **Residual Risk:** Post-mitigation score.

### C. AI Suggestions (`groqService.ts`)

- **Trigger:** Available in the Risk Form via "AI Suggest" button.
- **Input:** Description, Impact, Likelihood, Effect.
- **Output:** JSON containing `strategy` (Avoid, Transfer, Mitigate, Accept) and concrete `action` steps in Thai.

### D. Dashboard & Analytics (`App.tsx`)

- **Filters:** Filter by Project or textual search.
- **Visuals:**
    - **Risk Matrix:** Shows risk concentration (Heatmap).
    - **Baseline Comparison:** Overlays project risks against industry standards.
    - **Status Charts:** Open vs. Closed, Severe Risks count.

---

## 5. Data Dictionary (Key Types)

**RiskItem**

```ts
interface RiskItem {
  id: string;               // Unique UUID
  riskId: string;           // Human readable ID (e.g., R-001)
  projectNo: string;        // Project Identifier
  description: string;
  initialRisk: RiskScore;   // { impact: 1-5, likelihood: 1-5 }
  residualRisk: RiskScore;  // { impact: 1-5, likelihood: 1-5 }
  status: 'Open' | 'In Progress' | 'Closed';
  mitigationStrategy: 'A' | 'T' | 'M' | 'AC';
  history: RiskSnapshot[];  // Audit trail
  // ... timestamps and ownership fields
}
```

**UserProfile**

```ts
interface UserProfile {
  id: string;               // Firebase UID
  email: string;
  role: 'Admin' | 'User';
  assignedProjects: string[]; // List of ProjectNos user has access to
}
```

---

## 6. Setup & Installation

**Prerequisites:** Node.js (v18+ recommended)

1. **Install Dependencies:**

   ```
   npm install
   ```

2. **Environment Configuration:**
   Create a `.env.local` file in the root directory:

   ```
   GROQ_API_KEY=gsk_your_actual_api_key_here
   ```

   *Note: Firebase config is currently hardcoded or managed via `firebaseService.ts`. Ensure you have the correct Firebase project credentials if setting up a new environment.*

3. **Run Development Server:**

   ```
   npm run dev
   ```

   Access the app at `http://localhost:5173`.

4. **Build for Production:**

   ```
   npm run build
   ```

---

## 7. Current State & Known Issues

- **Source Location:** Main logic sits in `App.tsx` which is growing large. Recommendation to split into `layouts/DashboardLayout.tsx` and context providers (e.g., `RiskContext`, `AuthContext`).
- **Permissons:** Firestore rules must match the application logic (User role checks) to prevent "Permission Denied" errors.
- **Hardcoded Lists:** Industry types and Risk categories are defined in constants. Adding new ones requires code changes.

---

## 8. Deployment

The project is configured for **Firebase Hosting**.

1. **Login to Firebase CLI:**

   ```
   npx firebase login
   ```

2. **Deploy:**

   ```
   npm run build
   npx firebase deploy
   ```